



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Tecnologias de Sistemas Operativos

Ano Letivo de 2025/2026

Trabalho Prático - Grupo 10

Afonso Martins
a106931

Rodrigo Fernandes
a104175

Goncalo Castro
a107337

Maio, 2026

TSO

Índice

1. Introdução	1
2. Arquitetura do Serviço	1
3. Funcionalidades	2
3.1. Deduplicação de Blocos por Conteúdo (SHA-512)	2
3.2. Gestão de Ficheiros, Diretorias e Remoção	2
3.3. Cache de Páginas (<i>Page Cache</i>)	3
4. Avaliação de Desempenho	4
4.1. Execução dos Testes	4
4.2. Avaliação de Desempenho e Recursos (FIO)	4
4.2.1. Configuração	4
4.2.2. Resultados de I/O e Latência	4
4.2.3. Utilização de Recursos (FUSE)	4
4.3. Poupança de Espaço e Avaliação eBPF	5
4.3.1. Resultados de Armazenamento	5
4.3.2. Validação a nível do Kernel (eBPF)	5
4.4. Teste de Carga Real (Carga Manual)	6
4.5. Comparação Global (FUSE Base e FUSE com Deduplicação + Page Cache) ...	6
4.5.1. Desempenho de I/O: Throughput, IOPS e Latência	7
4.5.2. Utilização de Recursos: CPU e RAM	7
4.5.3. Armazenamento: Espaço Físico Ocupado no Backend	8
5. Conclusão	8
6. Anexos	9
6.1. Manual de Utilização	9
6.1.1. Pré-requisitos	9
6.1.2. Compilação	9
6.1.3. Montar o Sistema de Ficheiros	9
6.1.4. Utilização Básica	10
6.1.5. Executar a Bateria de Testes Completa	10
6.1.6. Compilar e Executar o Tracer eBPF Manualmente	10
6.1.7. Notas Importantes	11

1. Introdução

O presente relatório descreve o desenvolvimento e avaliação de um sistema de ficheiros com deduplicação de dados, implementado sobre a plataforma FUSE (*Filesystem in Userspace*) em Linux, no âmbito da Unidade Curricular de Tecnologias de Sistemas Operativos.

O objetivo central do trabalho é poupar espaço em disco através da técnica de deduplicação em blocos de tamanho fixo (4KB), eliminando cópias redundantes de dados antes de as persistir fisicamente. O sistema é desenhado para um modelo de utilização *append-only*, em que as escritas são sempre alinhadas e múltiplas de 4KB, como previsto no enunciado do trabalho prático.

A solução desenvolvida destaca-se em três componentes principais: (1) um sistema de ficheiros FUSE que interceta as operações de escrita e leitura em *user-space*; (2) um mecanismo de deduplicação baseado em *hashing* SHA-512 por bloco, com indexação em memória, persistência em disco e *caching*; e (3) um módulo de monitorização eBPF (*systracer*) que valida, ao nível do *kernel*, a efetiva redução de escritas físicas.

O relatório está organizado da seguinte forma: a Secção 2 descreve a arquitetura geral do sistema; a Secção 3 detalha as funcionalidades implementadas; a Secção 4 apresenta a avaliação experimental completa, incluindo comparação com um FUSE *passthrough* base; e a Secção 5 conclui com uma reflexão crítica sobre os resultados obtidos.

2. Arquitetura do Serviço

O sistema de ficheiros implementado assenta numa arquitetura de três camadas, ilustrada na **Figura 1**. A camada de interface com o utilizador é exposta através de FUSE, que monta um ponto de acesso lógico (por omissão em `codebase/tmp/meu_fuse/tmp`) onde o utilizador interage com ficheiros e diretórias de forma completamente transparente. Toda a escrita e leitura passa pelo *daemon FUSE* em *user-space*, que interceta cada pedido antes de o encaminhar para o armazenamento físico.

A camada intermédia é responsável pela lógica de deduplicação e pela gestão da *cache* em memória. Quando um bloco de 4KB é escrito, o programa calcula o seu **hash SHA-512** (tipo de *hash* usado em criptografia) e consulta uma tabela de indexação em memória (`htable`), que mapeia *hashes* para localizações físicas no *backend*. Se o bloco já existir, a escrita física é suprimida e apenas os metadados do ficheiro são atualizados com a referência ao bloco existente. Caso contrário, o bloco é persistido como um novo ficheiro na diretoria `/backend/.dedupblocks/`, nomeado pelo seu *hash* hexadecimal. Adicionalmente, o sistema integra uma *Page Cache* em *user-space* gerida com uma política LRU, que armazena os blocos mais recentes em memória para acelerar acessos subsequentes (leituras de blocos partilhados ou recém-escritos), evitando acessos desnecessários ao disco. Uma tabela de contagem de referências (`refcount_htable`) garante que um bloco físico só é eliminado quando nenhum ficheiro lógico o referencia, suportando assim a remoção correta de ficheiros.

A camada de persistência é composta pela diretoria `/backend`, que armazena dois tipos de estruturas: os blocos únicos em `.dedupblocks/` e o índice serializado `.dedupindex.bin`, escrito em disco no momento da desmontagem (`destroy`) e recarregado na montagem seguinte, garantindo a continuidade dos dados entre sessões. Os metadados de cada ficheiro lógico (tamanho, número

de blocos e lista ordenada de referências aos *hashes*) correspondentes são mantidos inteiramente em memória durante a execução, na forma de estruturas do tipo Filemeta.

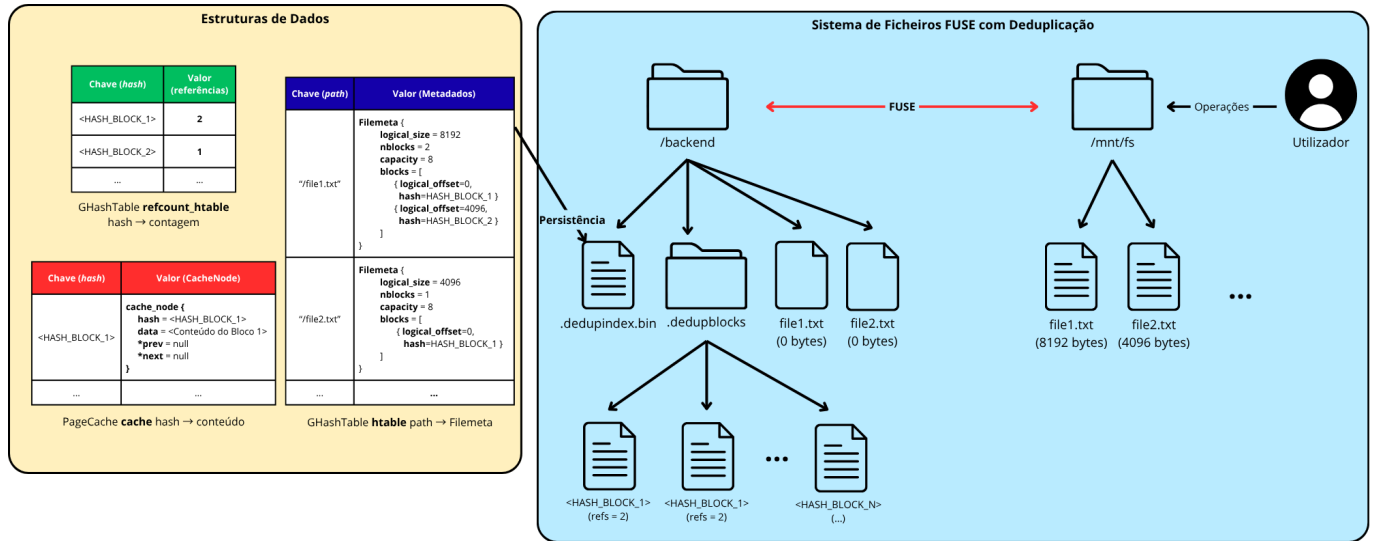


Figura 1: Arquitetura e Funcionamento geral do Sistema de Ficheiros implementado

3. Funcionalidades

3.1. Deduplicação de Blocos por Conteúdo (SHA-512)

A funcionalidade central do sistema é a deduplicação *inline* de blocos de 4KB, aplicada no caminho crítico de escrita, antes de qualquer dado ser persistido em disco. Cada pedido de escrita é dividido em blocos de exatamente 4096 bytes, e para cada bloco é calculado um *hash* SHA-512 que serve como identificador único do seu conteúdo.

O processo de escrita segue os seguintes passos:

1. Receção do pedido de escrita pelo *handler* FUSE;
2. Divisão dos dados em blocos de 4KB alinhados;
3. Cálculo do SHA-512 de cada bloco;
4. Consulta do *hash* na *htable*:
 - 4.1. Se o *hash* já existir, regista-se apenas a referência no Filemeta do ficheiro e incrementa-se o contador em *refcount_hhtable*, sem qualquer escrita física;
 - 4.2. Caso o *hash* seja novo, o bloco é escrito em **/backend/.dedupblocks/<hash_hex>** e indexado em ambas as tabelas. Na leitura, o sistema consulta o Filemeta do ficheiro para obter a sequência de *hashes* correspondente ao intervalo de *bytes* solicitado, e lê os blocos físicos respetivos do *backend*, reconstituindo os dados de forma transparente para o utilizador.

Esta abordagem garante que a deduplicação é inter-ficheiros: blocos com conteúdo idêntico em ficheiros distintos partilham a mesma cópia física, independentemente da sua origem.

3.2. Gestão de Ficheiros, Diretorias e Remoção

O sistema implementa todas as operações necessárias para um sistema de ficheiros funcional: criação, listagem e remoção de diretorias (*mkdir*, *readdir*, *rmdir*), bem como criação, leitura, escrita e remoção de ficheiros (*create*, *read*, *write*, *unlink* através dos usuais comandos *touch*, *cat*, *echo*, *rm*, ...). A listagem de diretorias (*readdir*) apresenta todos os ficheiros com o seu tamanho lógico correto (i.e., o número de blocos referenciados multiplicado por 4KB), indepen-

dentemente da partilha física de blocos com outros ficheiros, garantindo transparência total para o utilizador.

A remoção de ficheiros (`unlink`) é implementada com suporte a *garbage collection* de blocos órfãos: ao eliminar um ficheiro, o sistema percorre os *hashes* do seu Filemeta, decrementa os contadores em `refcount_hstable` e, para cada bloco cujo contador chega a zero, elimina o ficheiro físico correspondente em `/backend/.dedupblocks/`. Isto garante que o espaço libertado pela remoção de um ficheiro é efetivamente recuperado no *backend* e fica disponível para novas escritas, desde que o bloco em questão não seja referenciado por nenhum outro ficheiro.

3.3. Cache de Páginas (*Page Cache*)

Para reduzir a latência de leitura sem comprometer a correção do sistema, foi implementada uma *page cache* em memória com **política de substituição LRU** (*Least-Recently-Used*). A cache armazena até 1024 blocos de 4KB (4MB no total), configurável através da constante `CACHE_CAPACITY` em `pagecache.h`.

A estrutura interna combina uma `GHashTable` da GLib para pesquisa em $O(1)$ por *hash* SHA-512 com uma lista duplamente ligada que mantém a ordem de utilização recente: o nó mais recentemente acedido ocupa a cabeça da lista (*head*), enquanto o candidato a evicção reside na cauda (*tail*). O acesso concorrente é protegido por um *mutex* dedicado, independente do *mutex* do índice de metadados.

A cache intervém em dois momentos do caminho crítico de I/O:

- **Leitura (`xmp_read`):** antes de abrir qualquer ficheiro em `/backend/.dedupblocks/`, o sistema consulta a cache com o *hash* do bloco pretendido. Em caso de *hit*, os dados são copiados diretamente para o *buffer* do utilizador sem

qualquer acesso a disco. Em caso de *miss*, o bloco completo de 4KB é lido do *backend*, inserido na cache (*cache warming*) e o excerto solicitado é entregue ao utilizador.

- **Escrita (`xmp_write`):** imediatamente após a persistência de um bloco novo no *backend*, o bloco é também inserido

na cache. Isto garante que leituras subsequentes ao mesmo bloco comuns em padrões de *write-then-read* são servidas integralmente a partir de memória, eliminando o custo de uma abertura de ficheiro desnecessária.

A evicção é realizada de forma automática e transparente: quando a capacidade máxima é atingida, o bloco menos recentemente utilizado é removido da lista e da tabela, libertando espaço para o novo bloco sem intervenção da aplicação. Blocos partilhados entre múltiplos ficheiros (o caso central da deduplicação) beneficiam diretamente desta política, uma vez que um único *hit* na cache serve leituras de qualquer ficheiro que referencie esse bloco.

4. Avaliação de Desempenho

A avaliação do sistema de deduplicação foi realizada através de testes de *benchmarking* (utilizando o **FIO**) e cargas de trabalho reais, com o objetivo de medir o desempenho (*Throughput*, *IOPS* e *Latência*), a poupança de espaço de armazenamento e a utilização de recursos (CPU e RAM) pelo processo FUSE. Adicionalmente, utilizou-se **eBPF** para validar as chamadas ao sistema e a efetiva poupança de escrita física no disco.

4.1. Execução dos Testes

Os testes foram executados com recurso a um *script* automatizado (`script.sh`), que configurou o ambiente, montou o sistema de ficheiros, capturou as métricas (com `pidstat` e `ebpf`) e executou os testes de carga.

4.2. Avaliação de Desempenho e Recursos (FIO)

4.2.1. Configuração

Utilizou-se a ferramenta FIO com a `ioengine=psync`, escrevendo ficheiros com blocos de 4KB para um total de 100MB por cenário. Variou-se a percentagem de blocos duplicados gerados em 0%, 50% e 100%.

4.2.2. Resultados de I/O e Latência

Os resultados de *Throughput*, *IOPS* e *Latência* foram extraídos diretamente dos ficheiros JSON gerados pela ferramenta FIO (`dedupe_0.json`, `dedupe_50.json` e `dedupe_100.json`). Para cada nível de deduplicação, analisou-se a largura de banda de escrita (`write.bw`), as operações por segundo (`write.iops`) e a latência média (`write.lat_ns.mean`). Observa-se que a deduplicação melhora significativamente o desempenho global de escrita do sistema. Como blocos repetidos não necessitam de ser fisicamente escritos no disco, a latência diminui consideravelmente, aumentando o **Throughput** de forma visível e escalar.

Cenário	Throughput (MB/s)	IOPS	Latência Média (us)
0% Deduplicação	8.02	2052	484
50% Deduplicação	9.31	2384	416
100% Deduplicação	10.75	2751	360

4.2.3. Utilização de Recursos (FUSE)

Os dados relativos à utilização de recursos provêm dos relatórios do utilitário `pidstat` (`fuse_resources_*.txt`), que monitorizou o processo FUSE, segundo a segundo, durante as operações do FIO. Os valores da tabela correspondem à média do consumo de CPU (`%usr + %system`) e ao uso de memória RAM (`%MEM`). Durante os testes, foi monitorizado o consumo do processo em **user-space**. O consumo de CPU situa-se em torno de 81–84% de um **core**, justificado pelo cálculo contínuo de **hashes** SHA-512 para cada bloco de 4KB processado. Com maior taxa de deduplicação, o CPU decresce ligeiramente porque blocos duplicados apenas consultam o índice em memória, sem escrita física. A utilização de RAM mantém-se leve e estável.

Cenário	CPU (Média)	RAM (Média)
0% Deduplicação	~ 84%	~ 0.22%
50% Deduplicação	~ 83%	~ 0.18%

100% Deduplicação	~ 81%	~ 0.12%
-------------------	-------	---------

4.3. Poupança de Espaço e Avaliação eBPF

4.3.1. Resultados de Armazenamento

Os dados de armazenamento foram obtidos analisando diretamente o ponto de montagem do **backend** após cada teste. Utilizou-se o comando `ls /backend/.dedupblocks/ | wc -l` para contar o número exato de blocos únicos físicos e `du -sh /backend/.dedupblocks/` para calcular o volume de bytes efetivamente ocupados no disco (registados no ficheiro `disk_usage.txt`). Os dados obtidos atestam a eficácia absoluta do mecanismo de deduplicação **offline** em **chunks** fixos. A 100% de redundância forçada pelo FIO, apenas 1 único bloco de 4KB foi efetivamente guardado no espaço **backend**, poupando cerca de 99.99% de dados (100MB teóricos de teste).

Cenário	Blocos Únicos Físicos	Espaço Ocupado no Backend
0% Deduplicação	25600	105 MB
50% Deduplicação	12684	52 MB
100% Deduplicação	1	8.0 KB

4.3.2. Validação a nível do Kernel (eBPF)

Através do módulo eBPF desenvolvido (`systracer`), monitorizámos de forma ativa as chamadas ao sistema de I/O e a transferência de **bytes** efetuada pelo Kernel.

Para chegar aos resultados analíticos, o script processou os relatórios de BPF intercetando a `syscall write` enviada pelo processo FUSE para o VFS (Layer 0), em comparação com as operações de escrita validadas pela camada nativa EXT4 do **backend** (Layer 2). O somatório destes *bytes* está sumarizado no ficheiro `disk_usage.txt`.

Nota Técnica sobre Arquitetura I/O: Avaliou-se o tráfego na fronteira VFS/EXT4 em detrimento da camada inferior de Disco Físico (**Block Device**). Isto acontece porque, por omissão, as instâncias FUSE atuam como uma camada intermédia (**buffered**) entre a aplicação e o sistema nativo. Mesmo quando a aplicação testa escritas diretas (ex: `direct=1` no FIO), o daemon FUSE não força a flag `O_DIRECT` na sua ligação ao EXT4. Consequentemente, as escritas são depositadas em RAM (no **Page Cache** do kernel) e posteriormente descarregadas assincronamente para o disco por **threads** internas de sistema (como os `kworkers`), perdendo assim o PID do nosso processo FUSE. Monitorizar no ponto exato de entrega ao EXT4 garante a recolha isolada e precisa do nosso tráfego.

Nos cenários com elevada deduplicação, é constatóvel o impacto no Kernel: apesar do FUSE (como aplicação) instruir escritas na ordem dos megabytes (VFS), o volume de tráfego intercetado na camada de ficheiros inferior (**EXT4**) denota uma queda dramática das escritas (de ~104MB em 0% de deduplicação para apenas 4KB em 100% de deduplicação), provando de forma independente os ganhos reais gerados pelo nosso módulo.

```
Blocos únicos: 25600
Espaço usado: 105M /backend/.dedupblocks/
Resumo de I/O BPF (Layer 0=Syscall, Layer 2=EXT4):
  VFS Escrito (Bytes): 108232704
  Block/EXT4 Escrito (Bytes): 104853504

Blocos únicos: 12684
Espaço usado: 52M /backend/.dedupblocks/
Resumo de I/O BPF (Layer 0=Syscall, Layer 2=EXT4):
  VFS Escrito (Bytes): 55365632
  Block/EXT4 Escrito (Bytes): 51941376

Blocos únicos: 1
Espaço usado: 8.0K /backend/.dedupblocks/
Resumo de I/O BPF (Layer 0=Syscall, Layer 2=EXT4):
  VFS Escrito (Bytes): 3485696
  Block/EXT4 Escrito (Bytes): 4096

Blocos únicos manuais: 2560
Espaço usado manual: 11M /backend/.dedupblocks/
```

Figura 2: Registo de validação de chamadas de sistema efetuadas via **tracer** eBPF

4.4. Teste de Carga Real (Carga Manual)

Por fim, como requisitado na avaliação, desenvolveu-se uma carga de trabalho independente simulando uso real: procedeu-se à geração de um ficheiro aleatório de 10MB utilizando a ferramenta nativa `dd`, seguida da cópia imediata (`cp`) do ficheiro duas vezes, simulando um cenário real de duplicação por utilizador (totalizando ~30MB em VFS). O FUSE respondeu corretamente à carga de trabalho de nível OS, criando apenas **2560 blocos únicos** correspondentes aos precisos 10MB do primeiro ficheiro gerado, mantendo a diretoria **backend** contida nos cerca de **11MB**. Fica assim provada a eficácia de poupança no uso do sistema de ficheiros em situações do “dia-a-dia” independentes de testes em **loop** sintético.

4.5. Comparação Global (FUSE Base e FUSE com Deduplicação + Page Cache)

Para validar de forma rigorosa os custos computacionais da implementação (**overhead**), executou-se exatamente a mesma bateria de testes FIO sobre uma instância padrão de FUSE **passthrough** (sem qualquer lógica de deduplicação), e também sobre a nossa implementação de deduplicação utilizando escritas com buffer ativo, permitindo assim acionar e testar o **Page Cache** do sistema em conjunto com o FUSE. Todos os cenários utilizaram o mesmo diretoria / **backend**, tamanho de bloco (4KB) e carga total (100MB). Esta abordagem permite comparar o impacto trilateral: FUSE direto (s/ dedup), Dedup FUSE (c/ I/O direto) e Dedup FUSE (c/ Page Cache).

4.5.1. Desempenho de I/O: Throughput, IOPS e Latência

O FUSE base limita-se a encaminhar chamadas diretamente para o EXT4, sem overhead de hashing por bloco. O nosso sistema, mesmo no pior caso (0% de deduplicação), processa cada bloco de 4KB com SHA-512 antes de escrever, o que justifica a diferença de Throughput e Latência.

Programa	Cenário	Throughput (MB/s)	IOPS	Latência (us)
FUSE Base	0% Deduplicação	36.52	9350	104
	50% Deduplicação	35.62	9120	106
	100% Deduplicação	37.17	9517	102
Dedup FUSE	0% Deduplicação	8.02	2052	484
	50% Deduplicação	9.31	2384	416
	100% Deduplicação	10.75	2751	360
Dedup FUSE (Page Cache)	0% Deduplicação	8.16	2088	475
	50% Deduplicação	9.59	2456	404
	100% Deduplicação	10.97	2809	353

Apesar do **overhead** de hashing, note-se que o sistema com deduplicação melhora de forma consistente com o aumento da taxa de duplicação (de 8.02 MB/s a 0% para 10.75 MB/s a 100%), dado que blocos já conhecidos são ignorados na escrita física. A variante com **Page Cache** apresenta um desempenho marginalmente superior ao Dedup FUSE base em todos os cenários (ex.: 8.16 vs 8.02 MB/s a 0%), porque blocos recém-escritos são inseridos diretamente na cache, eliminando o custo de abertura de ficheiro em leituras subsequentes ao mesmo bloco. O FUSE base, por sua vez, escreve sempre todos os blocos sem qualquer lógica de deduplicação, alcançando throughput de escrita ~4.5× superior, mas sem qualquer poupança de espaço.

4.5.2. Utilização de Recursos: CPU e RAM

Programa	Cenário	CPU (Média)	RAM (Média)	Overhead CPU
Dedup FUSE	0% Deduplicação	~ 84%	~ 0.22%	+36%
	50% Deduplicação	~ 83%	~ 0.18%	+35%
	100% Deduplicação	~ 81%	~ 0.12%	+33%
Dedup FUSE (Page Cache)	0% Deduplicação	~ 84%	~ 0.22%	+36%
	50% Deduplicação	~ 82%	~ 0.20%	+34%
	100% Deduplicação	~ 80%	~ 0.12%	+32%
FUSE Base	0% Deduplicação	~ 48%	~ 0.03%	—
	50% Deduplicação	~ 48%	~ 0.03%	—
	100% Deduplicação	~ 48%	~ 0.02%	—

O custo de CPU é o aspeto mais relevante da comparação. O cálculo contínuo de **hashes** SHA-512 por cada bloco de 4KB representa ~32–36% de CPU adicional face ao FUSE base, ocupando consistentemente cerca de 80–84% de um **core** no sistema de deduplicação contra ~48% no FUSE base. Em compensação, o uso de RAM é superior no sistema de deduplicação (~0.12–0.22% vs ~0.02–0.03%), dado que o índice de blocos e os metadados de ficheiros residem inteiramente em memória. A variante com **Page Cache** apresenta consumo de RAM e CPU praticamente idêntico ao Dedup FUSE, correspondendo à cache LRU de 4MB pré-alocada — estrutura negligenciável face ao índice em memória.

4.5.3. Armazenamento: Espaço Físico Ocupado no Backend

A diferença mais expressiva entre os dois sistemas reside no uso de armazenamento. O FUSE base escreve sempre os 100MB completos para o /backend, independentemente do grau de redundância dos dados. O sistema de deduplicação, ao detetar blocos repetidos, recusa-os e evita a escrita física duplicada.

Programa	Cenário	Espaço Físico	Poupança
Dedup FUSE	0% Deduplicação	105 MB	0%
	50% Deduplicação	52 MB	~ 50%
	100% Deduplicação	8.0 KB	~ 99.99%
Dedup FUSE (Page Cache)	0% Deduplicação	105 MB	0%
	50% Deduplicação	52 MB	~ 50%
	100% Deduplicação	8.0 KB	~ 99.99%
FUSE Base	0% Deduplicação	101 MB	—
	50% Deduplicação	101 MB	—
	100% Deduplicação	101 MB	—

Na carga manual (10MB + 2 cópias), o FUSE base armazenou os 3 ficheiros separadamente, ocupando ~31MB no total. O sistema de deduplicação reduziu esse volume para ~11MB (apenas os blocos únicos do ficheiro original), comprovando eficácia em cenários reais de duplicação por utilizador.

A lógica de deduplicação introduz um **overhead** de CPU e latência face ao FUSE base, justificado pelo custo computacional do SHA-512 por bloco. Este custo reverte-se numa poupança de armazenamento extrema e escalável — de ~50% com dados 50% redundantes a ~99.99% com dados totalmente duplicados — demonstrando que o compromisso desempenho/espço é amplamente favorável em sistemas com elevada redundância de dados.

5. Conclusão

O trabalho desenvolvido demonstrou que é possível implementar um sistema de ficheiros com deduplicação de dados eficaz e transparente sobre a plataforma FUSE, com ganhos de armazenamento expressivos e um *overhead* computacional controlado.

Os resultados da avaliação experimental confirmam que o sistema cumpre os objetivos propostos. Em cenários com elevada redundância de dados (100% de duplicação), a poupança de armazenamento atingiu 99.99%, reduzindo 100MB de escrita lógica para um único bloco físico de 4KB no *backend*. Mesmo em cenários com 50% de redundância, a poupança foi proporcional (~50%), demonstrando a linearidade e previsibilidade do mecanismo.

O principal custo da implementação é o *overhead* de CPU introduzido pelo cálculo contínuo de *hashes* SHA-512 por bloco: o sistema consome cerca de 32–36% de CPU adicional face ao FUSE *passthrough* base, ocupando ~80–84% de um *core* contra ~48% no FUSE base. Este custo reflete-se numa redução de *Throughput* de escrita de ~4.5× face ao FUSE base no pior caso (0% de deduplicação), passando de 36.52 MB/s para 8.02 MB/s. Contudo, este *trade-off* é amplamente justificado em sistemas orientados ao arquivo de dados, onde a redundância é elevada e a poupança de espaço tem maior impacto do que a latência de escrita.

A validação independente via eBPF corroborou os resultados obtidos por medição direta: o *tracer* *systracer* confirmou a queda dramática do tráfego efetivo na fronteira VFS/EXT4 (de ~104MB para 4KB em 100% de deduplicação), provando que a poupança não é apenas aparente ao nível lógico, mas real ao nível das escritas no sistema de ficheiros nativo.

Como trabalho futuro, seria interessante explorar a paralelização do cálculo de *hashes* (por exemplo, usando múltiplas *threads* ou instruções SHA-NI do processador), bem como avaliar estratégias de compressão complementar dos blocos físicos. A tolerância a falhas súbitas através de um *write-ahead log* para o índice de deduplicação seria também uma melhoria relevante para cenários de produção.

6. Anexos

6.1. Manual de Utilização

Este manual descreve os passos necessários para compilar, montar e testar o sistema de ficheiros FUSE com deduplicação, bem como o FUSE base para fins de comparação.

6.1.1. Pré-requisitos

Antes de começar, garante que os seguintes pacotes estão instalados no sistema:

```
# Ubuntu/Debian
sudo apt install gcc make libfuse3-dev libglib2.0-dev \
    libssl-dev libbz2-dev fio sysstat

# Para o tracer eBPF (necessita de kernel >= 5.8 com BTF)
sudo apt install libbpf-dev clang llvm linux-tools-$(uname -r)
```

Adicionalmente, é necessário criar o diretoria `/backend` que serve como armazenamento físico do sistema:

```
sudo mkdir -p /backend
sudo chown $USER:$USER /backend
```

6.1.2. Compilação

Todos os binários são compilados a partir da pasta `codebase/` com um único comando:

```
cd codebase/
make
```

Isto produz dois executáveis:

- `passthrough` FUSE com deduplicação por blocos SHA-512
- `passthrough_normal` FUSE **passthrough** base, sem deduplicação (para comparação)

Para compilar apenas um dos binários individualmente:

```
make passthrough          # apenas o sistema de deduplicação
make passthrough_normal  # apenas o FUSE base
```

Para limpar os artefactos de compilação:

```
make clean
```

6.1.3. Montar o Sistema de Ficheiros

Cria o ponto de montagem e monta o FUSE com deduplicação:

```
mkdir -p codebase/tmp/meu_fuse/tmp
```

```
# FUSE com deduplicação (em foreground para ver output de debug)
```

```
./codebase/passthrough -f codebase/tmp/meu_fuse/tmp
```

```
# FUSE base sem deduplicação (usa módulo subdiretório para mapear para /backend)
```

```
./codebase/passthrough_normal -f \  
-o modules=subdiretório,subdiretório=/backend \  
codebase/tmp/meu_fuse/tmp
```

Para desmontar:

```
fusermount3 -u codebase/tmp/meu_fuse/tmp
```

6.1.4. Utilização Básica

Com o FUSE montado, o ponto de montagem comporta-se como um diretório normal. Todos os arquivos escritos são automaticamente deduplicados por blocos de 4KB:

```
MOUNT=codebase/tmp/meu_fuse/tmp
```

```
# Criar um arquivo
```

```
dd if=/dev/urandom of=$MOUNT/ficheiro_base.bin bs=1M count=10
```

```
# Copiar (os blocos duplicados são detectados e não reescritos)
```

```
cp $MOUNT/ficheiro_base.bin $MOUNT/copia1.bin
```

```
cp $MOUNT/ficheiro_base.bin $MOUNT/copia2.bin
```

```
# Ver quantos blocos únicos foram guardados no backend
```

```
ls /backend/.dedupblocks/ | wc -l # deve mostrar ~2560 (10MB / 4KB)
```

```
du -sh /backend/.dedupblocks/ # ~10MB em vez de ~30MB
```

6.1.5. Executar a Bateria de Testes Completa

O script `tests/script.sh` executa automaticamente testes FIO com 0%, 50% e 100% de deduplicação, recolhendo métricas de I/O, CPU/RAM e espaço em disco:

```
cd tests/
```

```
chmod +x script.sh
```

```
./script.sh
```

Os resultados são guardados em `tests/fio_results/`:

- `dedupe_0.json`, `dedupe_50.json`, `dedupe_100.json` métricas FIO (IOPS, Throughput, Latência)
- `fuse_resources_0.txt`, `fuse_resources_50.txt`, `fuse_resources_100.txt` CPU e RAM via `pidstat`
- `bpf_0.txt`, `bpf_50.txt`, `bpf_100.txt` syscalls interceptadas pelo tracer eBPF
- `disk_usage.txt` contagem de blocos únicos e espaço ocupado

Para executar o teste equivalente sobre o FUSE base (sem deduplicação):

```
chmod +x script_normal.sh
```

```
./script_normal.sh
```

Os resultados do FUSE base são guardados com o prefixo `normal_` na mesma pasta.

6.1.6. Compilar e Executar o Tracer eBPF Manualmente

O tracer `systracer` pode ser executado de forma independente para monitorizar qualquer processo FUSE:

```

cd tests/BPF/
make

# Obter o PID do processo FUSE (após montagem)
FUSE_PID=$(pgrep -x passthrough)

# Iniciar o tracer (requer root)
sudo ./sysstracer $FUSE_PID > saida_bpf.txt 2>&1 &

# ... executar a carga de trabalho desejada ...

# Parar o tracer
sudo kill %1

```

6.1.7. Notas Importantes

- O diretoria /backend/.dedupblocks/ é criado automaticamente na primeira montagem do FUSE com deduplicação. Não deve ser apagado manualmente enquanto o FUSE estiver montado.
- O índice de deduplicação é persistido em /backend/.dedupindex.bin no momento em que o FUSE é desmontado (destroy). Se o processo for terminado abruptamente (kill -9), o índice pode ficar desatualizado.
- Para reiniciar o sistema a partir do zero (limpar todos os dados e metadados):

```

fusermount3 -u codebase/tmp/meu_fuse/tmp 2>/dev/null
sudo find /backend -mindepth 1 -delete

```

- O sistema requer user_allow_other ou execução com permissões adequadas.